

- 基本概念
- 基本数据类型
- 运算符
- 容器
- 类型转换
- 动态类型



# 基本概念

# Python 语句以行尾结束

可以把多条语句写在同一行（用 ; 分隔）

```
# two statements in a single  
l = 10; b = 5
```

也可以将一条语句拆分到多行：

使用反斜杠 (\) 将一条语句拆成两行

```
x = 1 + 2 + 3 + 4 + \  
    5 + 6 + 7 + 8
```

圆括号 () 可让表达式自然地跨多行书写

```
x = (1 + 2 + 3 + 4 +  
    5 + 6 + 7 + 8)
```

- 添加到代码中的人类可读注释或说明
- 在编译或解释过程中被忽略（不影响执行）
- 在 Python 中，注释以 # 开头；其后内容会被解释器忽略

## 单行注释

```
01. | # This is a single line comment
02. | x = 20
```

## 行内注释

```
01. | x = 20 # store 20 in x
```

## 多行注释

没有像 C 中 /\* \*/ 那样的独立写

```
01. | # This is a
02. | # multi-line
03. | # comment
04. | x = 20
```

- 关键字是 Python 中的保留字：例如 `import`、`True`、`False`、`def`、`return ...`
- 它们的含义由 Python 解释器预先定义
- 每个关键字都有特定的用途和行为
- 它们定义了 Python 代码的基本语法和结构
- 不能用作变量、函数、类等名称

# ■ 关键字

- 关键字集合可能随 Python 版本而变化

|        |          |         |          |        |
|--------|----------|---------|----------|--------|
| False  | await    | else    | import   | pass   |
| None   | break    | except  | in       | raise  |
| True   | class    | finally | is       | return |
| and    | continue | for     | lambda   | try    |
| as     | def      | from    | nonlocal | while  |
| assert | del      | global  | not      | with   |
| async  | elif     | if      | or       | yield  |

# 获取关键字列表

- 可以查看当前 Python 版本中可用的关键字

```
python3
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import keyword
>>> print(keyword.kwlist)
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
>>> help('keywords')

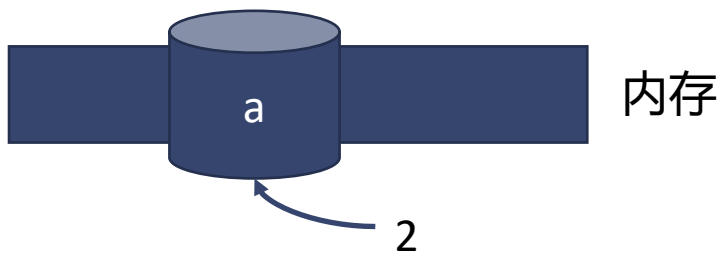
Here is a list of the Python keywords.  Enter any keyword to get more help.

False      class      from       or
None        continue  global     pass
True        def        if         raise
and         del        import     return
as          elif       in         try
assert      else       is         while
async       except     lambda     with
await       finally   nonlocal   yield
break       for        not

>>> █
```

- 变量是内存中保存值的命名存储位置
- 变量中存储的值可以在程序执行过程中改变
- 可以把它想象成带标签的盒子：可以放进去、取出来，或换成新的东西

简单理解：存放数据的容器





# ■ 声明变量并赋值

- 语法: `variable_name = value`
- 无需预先指定类型, 赋值时即创建

```
01. int main() {  
02.     int a = 1;  
03.     return 0;  
04. }
```

v.s.

```
01. a = 1  
02. print(type(a)) # <class 'int'>
```

C 代码

Python 代码

为变量赋新值时, 其类型会自动更新

```
01. import sys  
02. a = 1  
03. print(a, type(a), sys.getsizeof(a)) # 1 <class 'int'> 28  
04. a = 'dog'  
05. print(a, type(a), sys.getsizeof(a)) # c <class 'str'> 50
```

# 数据类型

- Python 中每个值都关联一种数据类型，它告诉解释器应如何处理该数据
- 数据类型决定了可执行哪些操作，以及哪些数据结构可以存储该值

## 常见数据类型

|     |          |
|-----|----------|
| 数字  | 布尔       |
| 字符串 | NoneType |
| 列表  | 元组       |
| 区间  | 字典       |
| 集合  |          |

# 函数

---

- 函数是具有特定名称、执行既定任务的可复用代码块。
- 它可以接收输入（称为实参）并返回输出（称为返回值）。



在 Python 中，函数主要分为两类：

- 内置函数：由 Python 提供，可直接使用（例如 `print()`、`len()`、`type()`）。
- 用户自定义函数：由程序员创建以完成自定义任务。

| Built-in Functions                                                                                                                                 |                                                                                                                                                              |                                                                                                         |                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A</b><br><code>abs()</code><br><code>aiter()</code><br><code>all()</code><br><code>any()</code><br><code>anext()</code><br><code>ascii()</code> | <b>E</b><br><code>enumerate()</code><br><code>eval()</code><br><code>exec()</code>                                                                           | <b>L</b><br><code>len()</code><br><code>list()</code><br><code>locals()</code>                          | <b>R</b><br><code>range()</code><br><code>repr()</code><br><code>reversed()</code><br><code>round()</code>                                                                                                   |
| <b>B</b><br><code>bin()</code><br><code>bool()</code><br><code>breakpoint()</code><br><code>bytearray()</code><br><code>bytes()</code>             | <b>F</b><br><code>filter()</code><br><code>float()</code><br><code>format()</code><br><code>frozenset()</code>                                               | <b>M</b><br><code>map()</code><br><code>max()</code><br><code>memoryview()</code><br><code>min()</code> | <b>S</b><br><code>set()</code><br><code>setattr()</code><br><code>slice()</code><br><code>sorted()</code><br><code>staticmethod()</code><br><code>str()</code><br><code>sum()</code><br><code>super()</code> |
| <b>C</b><br><code>callable()</code><br><code>chr()</code><br><code>classmethod()</code><br><code>compile()</code><br><code>complex()</code>        | <b>G</b><br><code>getattr()</code><br><code>globals()</code>                                                                                                 | <b>N</b><br><code>next()</code>                                                                         | <b>T</b><br><code>tuple()</code><br><code>type()</code>                                                                                                                                                      |
| <b>D</b><br><code>delattr()</code><br><code>dict()</code><br><code>dir()</code><br><code>divmod()</code>                                           | <b>H</b><br><code>hasattr()</code><br><code>hash()</code><br><code>help()</code><br><code>hex()</code>                                                       | <b>O</b><br><code>object()</code><br><code>oct()</code><br><code>open()</code><br><code>ord()</code>    | <b>V</b><br><code>vars()</code>                                                                                                                                                                              |
|                                                                                                                                                    | <b>I</b><br><code>id()</code><br><code>input()</code><br><code>int()</code><br><code>isinstance()</code><br><code>issubclass()</code><br><code>iter()</code> | <b>P</b><br><code>pow()</code><br><code>print()</code><br><code>property()</code>                       | <b>Z</b><br><code>zip()</code>                                                                                                                                                                               |
|                                                                                                                                                    |                                                                                                                                                              |                                                                                                         | <code>__import__()</code>                                                                                                                                                                                    |

# 用 input() 输入

内置函数 input() 用于读取用户输入

**input(prompt)**

- **Prompt:** 在等待输入前显示提示信息
- **返回类型:** 始终返回字符串
- **类型转换:** 使用类型转换 (例如 float()) 得到其他数据类型

```
01. number = input('Enter a number: ')
02. # Enter a number: 2
03. print(number, type(number))
04. # 2 <class 'str'>
05. number_int = int(number)
06. print(number_int, type(number_int))
07. # 2 <class 'int'>
08. number_float = float(number)
09. print(number_float, type(number_float))
10. # 2.0 <class 'float'>
```



# 基本数据类型

# 为什么基本数据类型很重要？

---

- 它们是 Python 中所有数据的基本构建块。
- 每一条信息，从简单计数到复杂数据结构，最终都由这些核心类型组成。
- 掌握它们是进行计算、做出逻辑判断、并在程序中表示信息的第一步。

Python 中的三种数值类型:

- int: 整数值 (例如 5)
- float: 浮点数值 (例如 3.14)
- complex: 复数 (例如 1+2j) , 实部 + 虚部

## Examples:

```
1 type(5)           # int
2 type(5.0)         # float
3 type(1+2j)        # complex
```



使用 bool 数据类型表示布尔值（True、False）。

通常是比较运算符的结果。

广泛用于条件语句和循环。

## Examples:

```
1 print(5 > 3)           # True
2 print(bool(0))         # False
3 print(bool("hi"))      # True
```

## 练习：基本数据类型

预测下列代码的输出。每个结果的数据类型是什么？

```
1 # Prediction 1
2 a = 15
3 print(type(a))
4
5 # Prediction 2
6 b = a / 2
7 print(type(b))
8
9 # Prediction 3
10 c = " "
11 print(bool(c))
12
13 # Prediction 4
14 d = 0.0
15 print(bool(d))
```



运算符

## ■ 为什么运算符很重要？

---

- 运算符是编程中的“动词”；它们让你操作并作用于数据。
- 没有它们，你虽能存储信息，却无法进行计算、比较或组合逻辑条件。
- 它们是构建表达式、实现算法的基本工具。

- 对操作数执行特定运算并返回结果的特殊符号
  - 一元：对一个操作数运算：  $-a$
  - 二元：对两个操作数运算：  $a + b$
- 类别：
  - 算术运算符
  - 关系运算符
  - 逻辑运算符
  - 位运算符
  - 赋值运算符
  - 成员运算符
  - 身份运算符

# ■ 算术运算符

- 7 种基本二元算术运算符，2 种一元运算符。

| Operator | Name             | Description                                    |
|----------|------------------|------------------------------------------------|
| $a + b$  | Addition         | Adds two operands                              |
| $a - b$  | Subtraction      | Subtracts second operand from first            |
| $a * b$  | Multiplication   | Multiplies two operands                        |
| $a / b$  | Division (float) | Divides first operand by second, returns float |
| $a // b$ | Floor division   | Divides and returns the floor of the quotient  |
| $a \% b$ | Modulus          | Returns the remainder after division           |
| $a ** b$ | Exponentiation   | Raises first operand to the power of second    |
| $-a$     | Unary minus      | Negate the operand                             |
| $+a$     | Unary plus       | Returns the same number (rarely used)          |

# 关系运算符

特点:

- 用于比较两个操作数的值
- 输出布尔值 (True 或 False)

| Operator   | Description   | Operator   | Description      |
|------------|---------------|------------|------------------|
| $a == b$   | Equal         | $a != b$   | Not equal        |
| $a < b$    | Less than     | $a > b$    | Greater than     |
| $a \leq b$ | Less or equal | $a \geq b$ | Greater or equal |

示例

```
01. 25 < 1 # False
02. 2 == 2 # True
03. 3 != 2 # True
```

将多个比较串联起来

```
01. 15 < 25 < 30 # True
02. 15 > 1 < 2 # True
03. 15 > 1 < 0 # False
```

# 逻辑运算符

- 用于用 "and"、"or" 和 "not" 组合布尔值

| Operator | Description                                                                                                         |
|----------|---------------------------------------------------------------------------------------------------------------------|
| and      | Returns <code>True</code> only if both operands are <code>True</code> , otherwise <code>False</code>                |
| or       | Returns <code>True</code> if at least one operand is <code>True</code> , otherwise <code>False</code>               |
| not      | Returns <code>True</code> if operand is <code>False</code> , and <code>False</code> if operand is <code>True</code> |

## Examples:

```
1 x = 4
2 (x < 6) and (x > 2) # True
3 (x > 10) or (x % 2 == 0) # True
4 not (x < 5) # False
```



## 位运算符

- 6 种按位运算符：作用于二进制表示

| Operator     | Name        | Description                           |
|--------------|-------------|---------------------------------------|
| $a \& b$     | Bitwise AND | Returns 1 if both bits are 1          |
| $a   b$      | Bitwise OR  | Returns 1 if at least one bit is 1    |
| $a \wedge b$ | Bitwise XOR | Returns 1 if bits are different       |
| $a \ll n$    | Left shift  | Shifts bits to left by $n$ positions  |
| $a \gg n$    | Right shift | Shifts bits to right by $n$ positions |
| $\sim a$     | Bitwise NOT | Inverts all bits                      |

## 按位运算符：二进制表示

- 二进制：使用 0 和 1 两种状态表示数字的二进制（base-2）数制

| $2^3$ | $2^2$ | $2^1$ | $2^0$ |
|-------|-------|-------|-------|
| 1     | 0     | 1     | 0     |

$$10 (1010) = 2^3 * 1 + 2^2 * 0 + 2^1 * 1 + 2^0 * 0$$

| $2^3$ | $2^2$ | $2^1$ | $2^0$ |
|-------|-------|-------|-------|
| 0     | 1     | 0     | 0     |

$$4 (100) = 2^2 * 1 + 2^1 * 0 + 2^0 * 0$$

# 按位运算符： 深入理解

理解按位运算：

- 用二进制表示这些数字
- 对每一位执行逻辑运算

示例：对 6 和 8 按位或得到 14

```
01. print(6 | 8)
```

用 bin() 观察二进制表示

```
01. print(bin(6))      # 0b110
02. print(bin(8))      # 0b1000
03. print(bin(14))     # 0b1110
```

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 0 |

## ■ 练习：按位运算符（二进制）

- $0001 \& 1111 = 0001$
- $1101 \& 0010 = 0000$
- $0010 \mid 1110 = 1110$
- $0100 \mid 1001 = 1101$
- $0010 \wedge 1101 = 1111$
- $1100 \wedge 1110 = 0010$
- $1100 \ll 1 = 11000$
- $1101 \gg 1 = 110$

| Operator     | Name        | Description                         |
|--------------|-------------|-------------------------------------|
| $a \& b$     | Bitwise AND | Returns 1 if both bits are 1        |
| $a \mid b$   | Bitwise OR  | Returns 1 if at least one bit is 1  |
| $a \wedge b$ | Bitwise XOR | Returns 1 if bits are different     |
| $a \ll n$    | Left shift  | Shifts bits to left by n positions  |
| $a \gg n$    | Right shift | Shifts bits to right by n positions |
| $\sim a$     | Bitwise NOT | Inverts all bits                    |

## ■ 练习：按位运算符（十进制）

- $7 \& 5$  ➤  $111 \& 101 = 101 = 5$
- $20 | 5$  ➤  $10100 | 101 = 10101 = 21$
- $20 \wedge 5$  ➤  $10100 \wedge 101 = 10001 = 17$
- $20 >> 3$  ➤  $10100 >> 3 = 10 = 2$
- $7 >> 1$  ➤  $111 >> 1 = 11 = 3$
- $20 << 2$  ➤  $10100 << 2 = 1010000 = 80$

| Operator     | Name        | Description                         |
|--------------|-------------|-------------------------------------|
| $a \& b$     | Bitwise AND | Returns 1 if both bits are 1        |
| $a   b$      | Bitwise OR  | Returns 1 if at least one bit is 1  |
| $a \wedge b$ | Bitwise XOR | Returns 1 if bits are different     |
| $a << n$     | Left shift  | Shifts bits to left by n positions  |
| $a >> n$     | Right shift | Shifts bits to right by n positions |
| $\sim a$     | Bitwise NOT | Inverts all bits                    |

# 赋值运算符

将运算与赋值结合的简写形式

| Operator  | Description  | Operator   | Description   |
|-----------|--------------|------------|---------------|
| $a+ = b$  | $a = a + b$  | $a- = b$   | $a = a - b$   |
| $a* = b$  | $a = a * b$  | $a// = b$  | $a = a // b$  |
| $a\% = b$ | $a = a \% b$ | $a* * = b$ | $a = a * * b$ |

$a (\cdot) = b$  等价于  $a = a (\cdot) b$

```
01. a = 3
02. a += 2
03. print(a) # 5
```

```
01. a = 3
02. a *= 4
03. print(a) # 12
```

## ■ ■ ■ 练习：赋值运算符

---

下列代码的输出是什么？

```
a = 1
a += 2
a * 4
a - 1
a /= 2
print(a)
```

✓ 0.0s

| Operator            | Description                                      |
|---------------------|--------------------------------------------------|
| <code>is</code>     | Test if two variables refer to the same object   |
| <code>is not</code> | Test if two variables refer to different objects |

## Examples:

```
1 a = [1, 2]
2 b = a
3 print(a is b)    # True
4 print(a == b)    # True
```



## 成员运算符

测试某个值或变量是否存在于集合中（例如字符串、列表、元组、集合和字典）

| Operator            | Description                          |
|---------------------|--------------------------------------|
| <code>in</code>     | Test if a value is in a sequence     |
| <code>not in</code> | Test if a value is not in a sequence |

```
01. 1 in [1, 2, 3]      # True
02. 1 in (1, 2, 3)     # True
03. 3 not in [1, 2, 3] # False
04. 'a' in 'abc'       # True
```

运算符的组合使用：

- $1 + 2 * 3 - 5 \% 2 = ?$
- $1 + 2 \& 5 * 2 = ?$

计算顺序是什么？

- 从高优先级到低优先级
- 从左到右

一些模式

- 算术 > 按位 > 比较 > 逻辑
- 一元 > 二元

| Operators                                                                                                                                                                                           | Meaning                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| <code>()</code>                                                                                                                                                                                     | Parentheses                                       |
| <code>**</code>                                                                                                                                                                                     | Exponent                                          |
| <code>+x</code> , <code>-x</code> , <code>~x</code>                                                                                                                                                 | Unary plus, Unary minus, Bitwise NOT              |
| <code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>                                                                                                                                  | Multiplication, Division, Floor division, Modulus |
| <code>+</code> , <code>-</code>                                                                                                                                                                     | Addition, Subtraction                             |
| <code>&lt;&lt;</code> , <code>&gt;&gt;</code>                                                                                                                                                       | Bitwise shift operators                           |
| <code>&amp;</code>                                                                                                                                                                                  | Bitwise AND                                       |
| <code>^</code>                                                                                                                                                                                      | Bitwise XOR                                       |
| <code> </code>                                                                                                                                                                                      | Bitwise OR                                        |
| <code>==</code> , <code>!=</code> , <code>&gt;</code> , <code>&gt;=</code> , <code>&lt;</code> , <code>&lt;=</code> , <code>is</code> , <code>is not</code> , <code>in</code> , <code>not in</code> | Comparisons, Identity, Membership operators       |
| <code>not</code>                                                                                                                                                                                    | Logical NOT                                       |
| <code>and</code>                                                                                                                                                                                    | Logical AND                                       |
| <code>or</code>                                                                                                                                                                                     | Logical OR                                        |

## 练习：运算符

result 的最终值是什么？逐步跟踪代码，特别注意运算符优先级与赋值

```
1  x = 10
2  y = 4
3  z = 2
4  # Step 1
5  x += y * z
6  # x is now?
7  # Step 2
8  y = x % 9
9  # y is now?
10 # Step 3
11 result = (x > 15) and (y == 0)
12 # What is result?
13 print(result)
```



容器

## ■ 为什么集合很重要？

---

- 基本数据类型一次只能保存一个值。集合（或数据结构）对于存储和组织相关数据组至关重要。
- 它们让你能管理复杂信息，例如学生列表、唯一元素集合，或单词到释义的映射。
- 为任务选择合适的集合是一项关键编程技能。

# ■ 可变与不可变

---

- Python 中有两类对象：可变与不可变
- 可变对象创建后可以被修改
  - list, dict, set
- 不可变对象创建后不能被修改
  - int, float, bool, str, tuple
- 可变对象不可哈希（即不能作为 dict 的键、set 的元素.....）

# 列表

## 特点

- 元素 (items) 的有序集合
- 用逗号分隔, 放在方括号 ([ ]) 内
- 值可以是任意数据类型
- 列表是可变的。

```
01. a = [1, 2, 5, 4]
02. print(type(a)) # <class 'list'>
```

```
01. # empty list
02. my_list = []
03.
04. # list of integers
05. my_list = [1, 2, 3]
06.
07. # list with mixed datatypes
08. my_list = [1, "Hello", [], 1.2]
```

# 元组

## 特点

- 有序的元素集合
- 用逗号分隔，放在圆括号 () 内
- 值可以是任意数据类型
- 元组不可变

```
01. a = (1, 2, 5, 4)
02. print(type(a)) # <class 'tuple'>
```

注意：真正重要的是逗号 (,)，而不是括号

```
01. a = 1, 2, 3
02. print(a, type(a)) # (1, 2, 3) <class 'tuple'>
```

可包含不同类型的元素

```
01. a = (1, 2, 5, 'dog', [], 6.7)
```



## 特点

- 用引号（单引号、双引号或三引号）括起的连续字符序列。
- 三引号用于定义多行字符串。
- 字符串不可变，创建后不能被修改

## Examples:

```
1 s1 = "Hello, World!"
2 s2 = """This is a
3 multi-line string."""
4 print(type(s1)) # <class 'str'>
```

# 字符串格式化

通过嵌入变量来格式化字符串的两种常见方式：

- `format()` 方法：使用花括号 `{}` 作为占位符，填入传入该方法的变量。
- 字符串格式化运算符 `%`：类似于 C 的 `printf()`，使用 `%s`（字符串）、`%d`（整数）等占位符。

## Examples:

```
1 language = "Python"
2 year = 1991
3 # Using the .format() method
4 print("{} , was developed in {}".format(language , year))
5 # Using the % operator
6 print("%s was developed in %d." % (language , year))
```

特点:

- 无序的元素集合
- 每个元素都是唯一的（无重复）。
- 用逗号分隔，放在花括号 {} 内。
- 集合元素必须是不可变（可哈希）类型。

**Examples:**

```
1  # A set of integers
2  my_set = {1, 2, 5, 4}
3  print(type(my_set))  # <class 'set'>
4  # Create an empty set with set()
5  empty_set = set()
6  # Note: {} creates an empty dictionary, not a set
```

## 集合是可变的

创建后可通过添加或删除元素进行修改。

添加元素：

- `add(elem)`：添加单个元素。
- `update(iterable)`：从可迭代对象（如列表或另一个集合）添加全部元素。重复项会自动丢弃。

删除元素：

- `remove(elem)`：删除一个元素；若元素不存在则抛出 `KeyError`。
- `discard(elem)`：若元素存在则删除；不存在则不做任何操作。

### Examples:

```
1 my_set = {1, 2, 3}
2 my_set.add(4) # {1, 2, 3, 4}
3 my_set.update([2, 5, 6]) # {1, 2, 3, 4, 5, 6}
```

## ■ 可哈希的集合? frozenset!

---

- 不可变（因此可哈希）的集合
- `frozenset()`: 内置函数，接受可迭代对象作为参数并使其不可变
- 用途：用作字典的键
- 支持集合运算（增删方法除外）

```
>>> a = {1, 2, 3}
>>> b = frozenset(a)
>>> b
frozenset({1, 2, 3})
>>> c = {b}
>>> c
{frozenset({1, 2, 3})}
```

## 特点:

- 一组键值对的集合（可变）。
- 键必须唯一，且为不可变（可哈希）类型。
- 值可以是任意数据类型，且允许重复。
- 用花括号 {} 括起，键值对之间用冒号 : 分隔

## Examples:

```
1 # A dictionary of student ages
2 person_age = {"John": 21, "Kim": 35, "Alex": 26}
3 print(type(person_age)) # <class 'dict'>
4 # Creating an empty dictionary
5 empty_dict = {}
```

# 访问与修改字典

## 访问值:

- 使用方括号 ['key'] 访问值。若键不存在会引发 KeyError。
- 使用 get('key') 方法; 若键不存在则返回 None (或默认值)。

## 添加或更新:

- 为新键赋值即可添加一对键值。
- 为已有键赋新值以更新它。

## Examples:

```
1 person_age = {"Kim": 35}
2 print(person_age['Kim'])      # 35
3 print(person_age.get('Alex')) # None
4 person_age['Steve'] = 32 # Adding a new element
5 # Updating an existing element
6 person_age['Kim'] = 30
7 print(person_age) # {'Kim': 30, 'Steve': 32}
```

## 通用集合方法

特点：

- 适用于一般集合，例如列表、元组和字符串。
- 返回新值，不修改原始集合。
- 比写循环更易读、更简洁

| Method                | Description                          |
|-----------------------|--------------------------------------|
| <code>all()</code>    | Judge if all element is True         |
| <code>any()</code>    | Judge if any element is True         |
| <code>len()</code>    | Return the length of a collection    |
| <code>max()</code>    | Return the maximum of a collection   |
| <code>min()</code>    | Return the minimum of a collection   |
| <code>sum()</code>    | Return the sum of a collection       |
| <code>sorted()</code> | Return a sorted list of a collection |





# 类型转换

## 为什么类型转换很重要？

---

- 数据并不总是以你需要的格式出现。类型转换是处理与准备数据的关键技能。
- 这对将用户通过 `input()` 得到的输入（始终是字符串）转换为数字以进行计算等任务至关重要。
- 这也让你能利用不同数据类型的特性，例如将列表转为集合以方便找出唯一元素。

# 类型转换

将对象从一种数据类型转换为另一种的过程。

隐式转换（强制转换）：

- Python 会自动转换数据类型以避免数据丢失。
- 例如，将 int 与 float 相加会得到 float。

显式转换（强制类型转换）：

- 由程序员使用 int()、float()、str()、list() 等内置函数手动转换

**Examples:**

```
1  # Implicit conversion
2  c = 2 + 4.5; print(type(c)) # <class 'float'>
3  # Explicit conversion
4  a = int(3.9); print(a)      # <class 'int'>
5  s = str(123); print(type(s)) # <class 'str'>
```

# 常见的显式类型转换

| Function                 | Description                        | Example                                                  |
|--------------------------|------------------------------------|----------------------------------------------------------|
| <code>int()</code>       | Convert to integer                 | <code>int('123')</code> \$to\$ 123                       |
| <code>float()</code>     | Convert to float                   | <code>float('123.4')</code> \$to\$ 123.4                 |
| <code>tuple()</code>     | Convert to tuple                   | <code>tuple([1,2,3])</code> \$to\$ (1,2,3)               |
| <code>set()</code>       | Convert to set (remove duplicates) | <code>set([1,2,2,3])</code> \$to\$ 1,2,3                 |
| <code>dict()</code>      | Create dict from key-value pairs   | <code>dict([('a',1),('b',2)])</code> \$to\$ 'a':1, 'b':2 |
| <code>complex()</code>   | Create complex number              | <code>complex(2,3)</code> \$to\$ (2+3j)                  |
| <code>bool()</code>      | Convert to boolean                 | <code>bool(0)</code> \$to\$ False                        |
| <code>bytes()</code>     | Convert to bytes                   | <code>bytes('hello', 'utf-8')</code> \$to\$ b'hello'     |
| <code>frozenset()</code> | Create immutable set               | <code>,2,3frozenset([1,2,3])</code> \$to\$ frozenset(1)  |

## Special Notes:

```
1 # Dict conversion requires proper key-value pairs
2 dict(['key', 'value'], ('a', 1)) # Valid
3 dict(['ab', 'cd']) # ValueError: dictionary update
    sequence element #0 has length 2; 2 is not 2
```

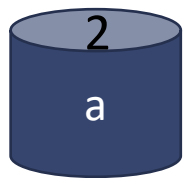


动态类型

# ■ 关于变量的问题

---

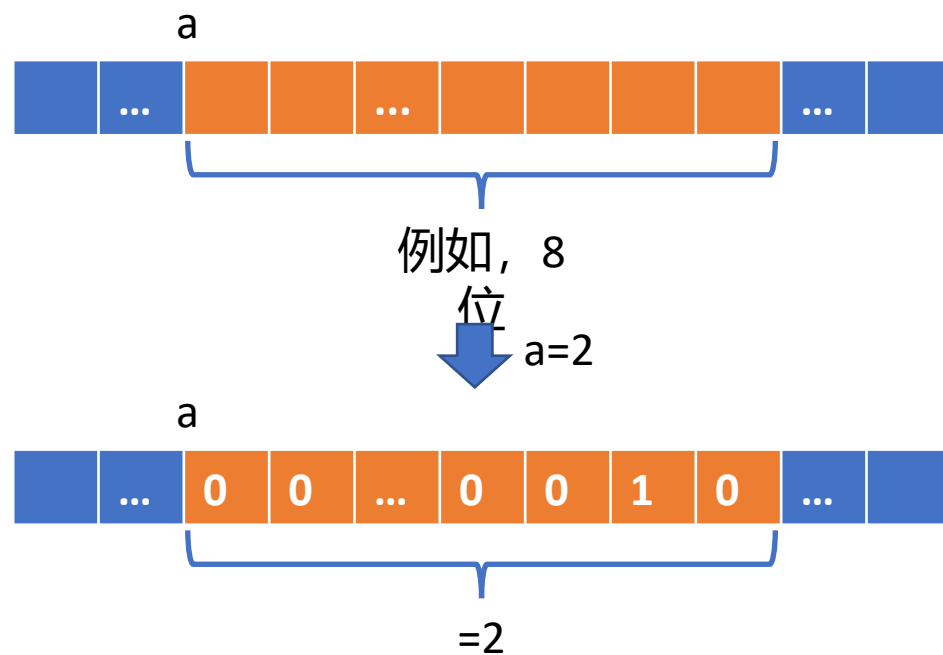
- 问题 1：变量如何保存数据？
- 问题 2：赋值过程中发生了什么？



# 基本思想

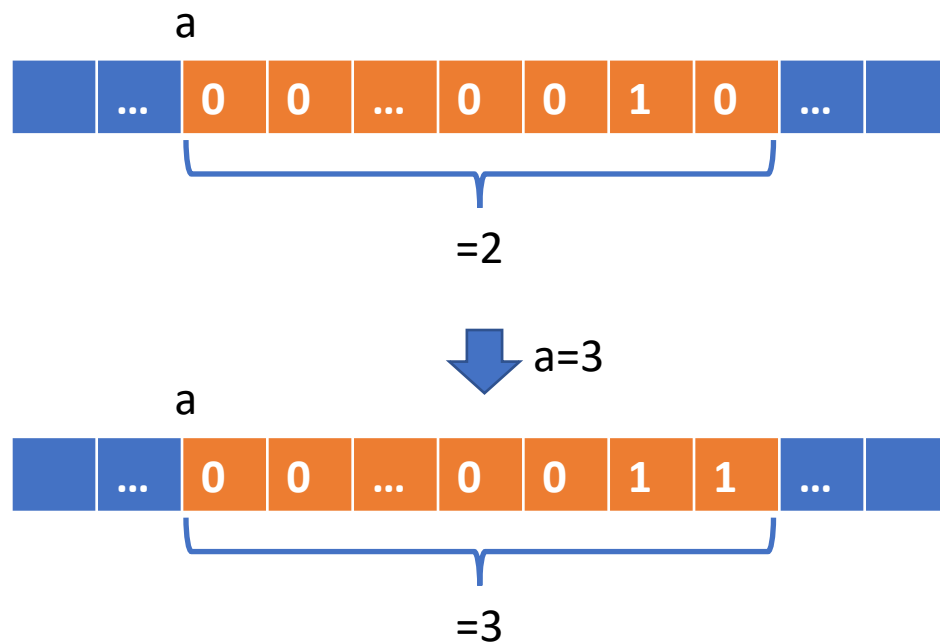
问题 1: 变量如何保存数据?

- 计算机为变量分配内存空间
- 所赋的值被写入该内存位置



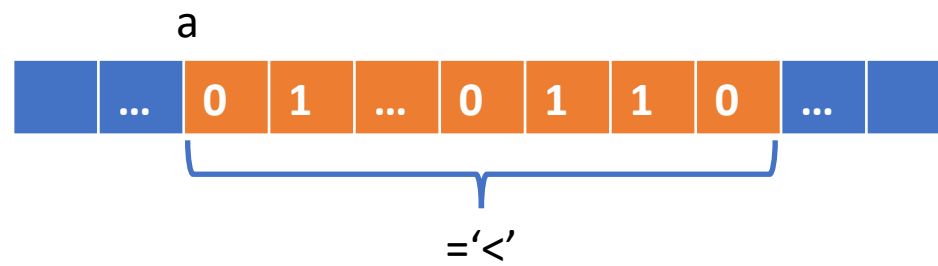
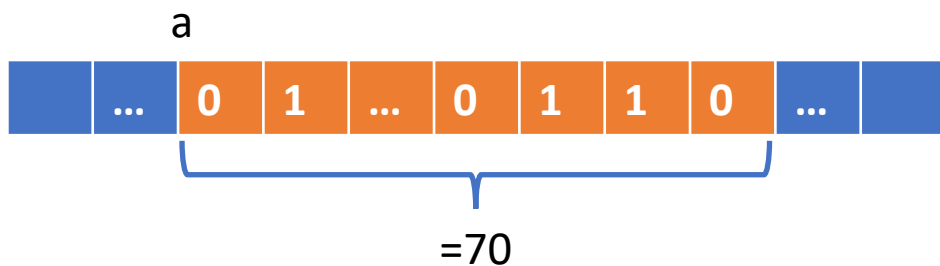
问题 2：赋值过程中发生了什么？

- 计算机清除内存中先前存储的值
- 然后将新值写入该内存位置



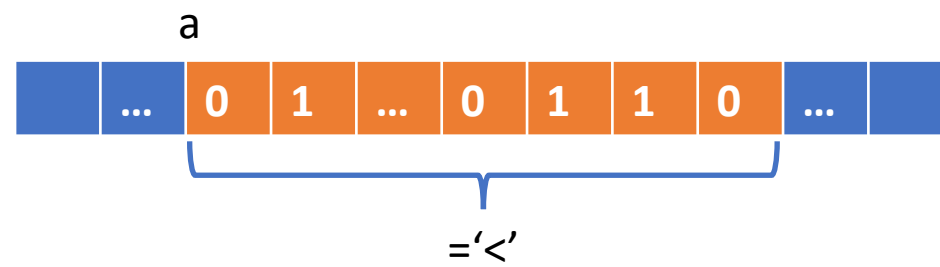
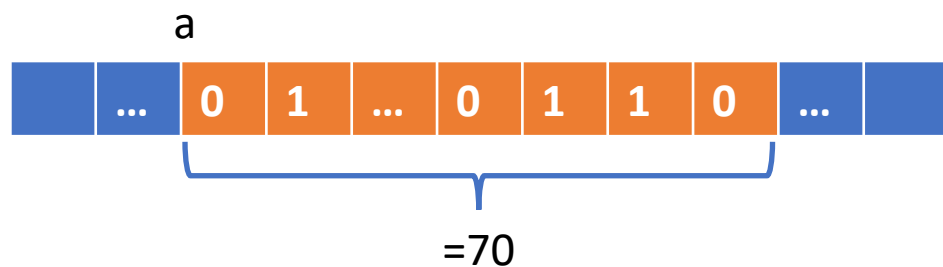


问题 3：类型信息是如何存储的？



同一份二进制数据可以表示不同类型

问题 3：类型信息是如何存储的？



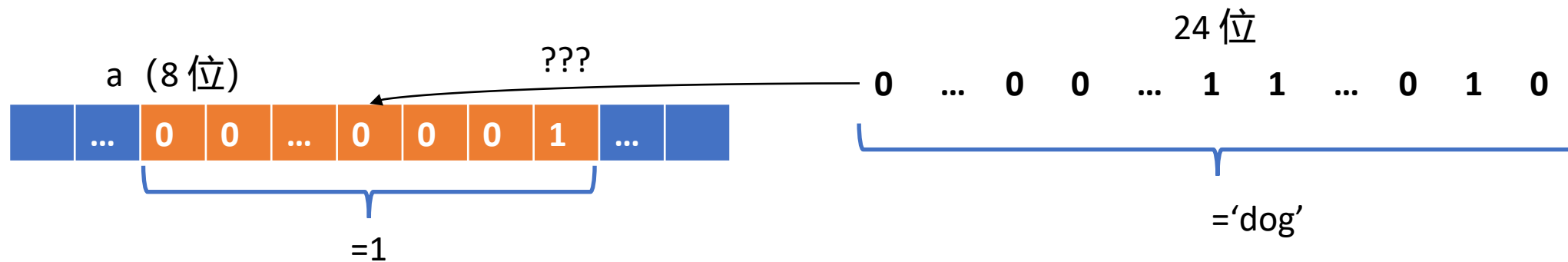
类型信息与变量相关联

# Python 中的动态类型

问题 4: Python 中的类型信息在哪里?

问题 5: 不同大小的值是如何存储的?

```
01. import sys
02. a = 1
03. print(a, type(a), sys.getsizeof(a)) # 1 <class 'int'> 28
04. a = 'dog'
05. print(a, type(a), sys.getsizeof(a)) # c <class 'str'> 50
```

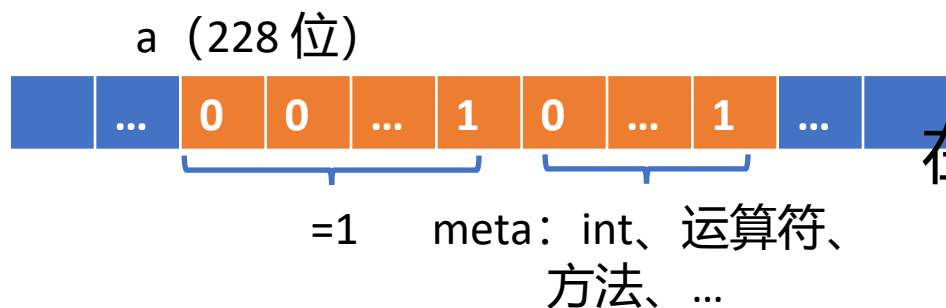


# Python 中的动态类型

问题 4: Python 中的类型信息在哪里?

- 类型信息附着在每个对象上, 而非变量名上

```
01. import sys
02. a = 1
03. print(a, type(a), sys.getsizeof(a)) # 1 <class 'int'> 28
04. a = 'dog'
05. print(a, type(a), sys.getsizeof(a)) # c <class 'str'> 50
```



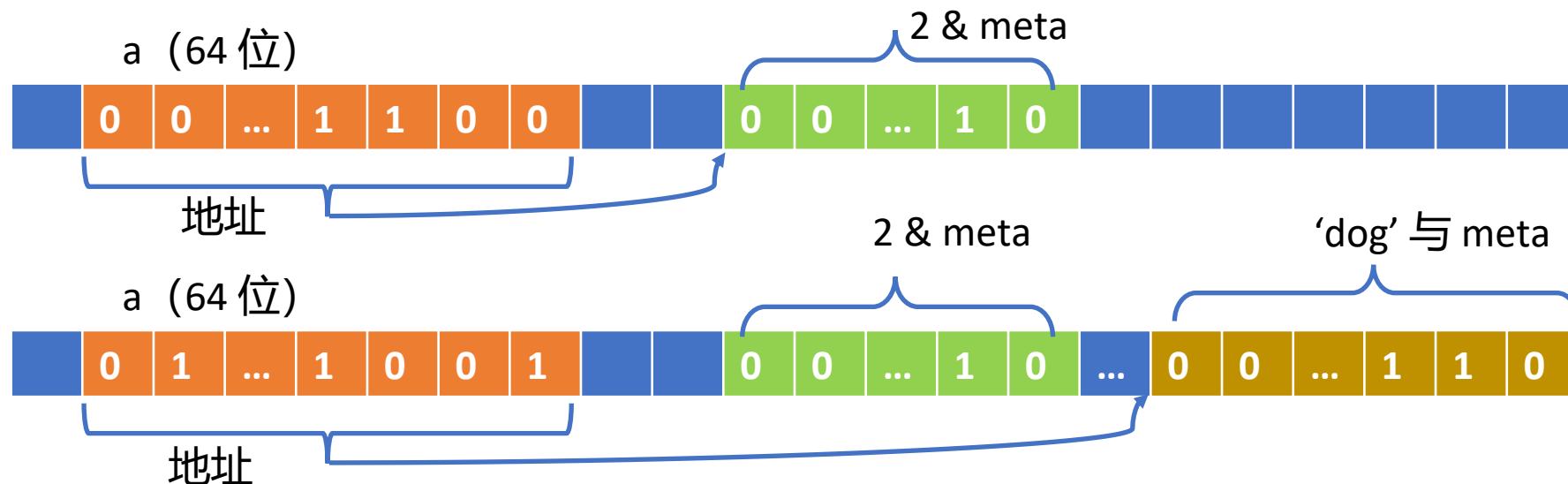
会带来额外的内存开销

在 C 中占 8 字节, 在 Python 中占 28 字节

# Python 中的动态类型

问题 5: 不同大小的值是如何存储的?

- 实际值存储在内存的其他位置
- 变量只保存对该值的引用 (地址)
- 赋新值会更新引用, 使其指向不同的地址



- `id()` 函数: 返回对象的唯一标识 (在 CPython 中对应对象的内存地址)

问题 6: 把一个变量赋给另一个变量会发生什么?

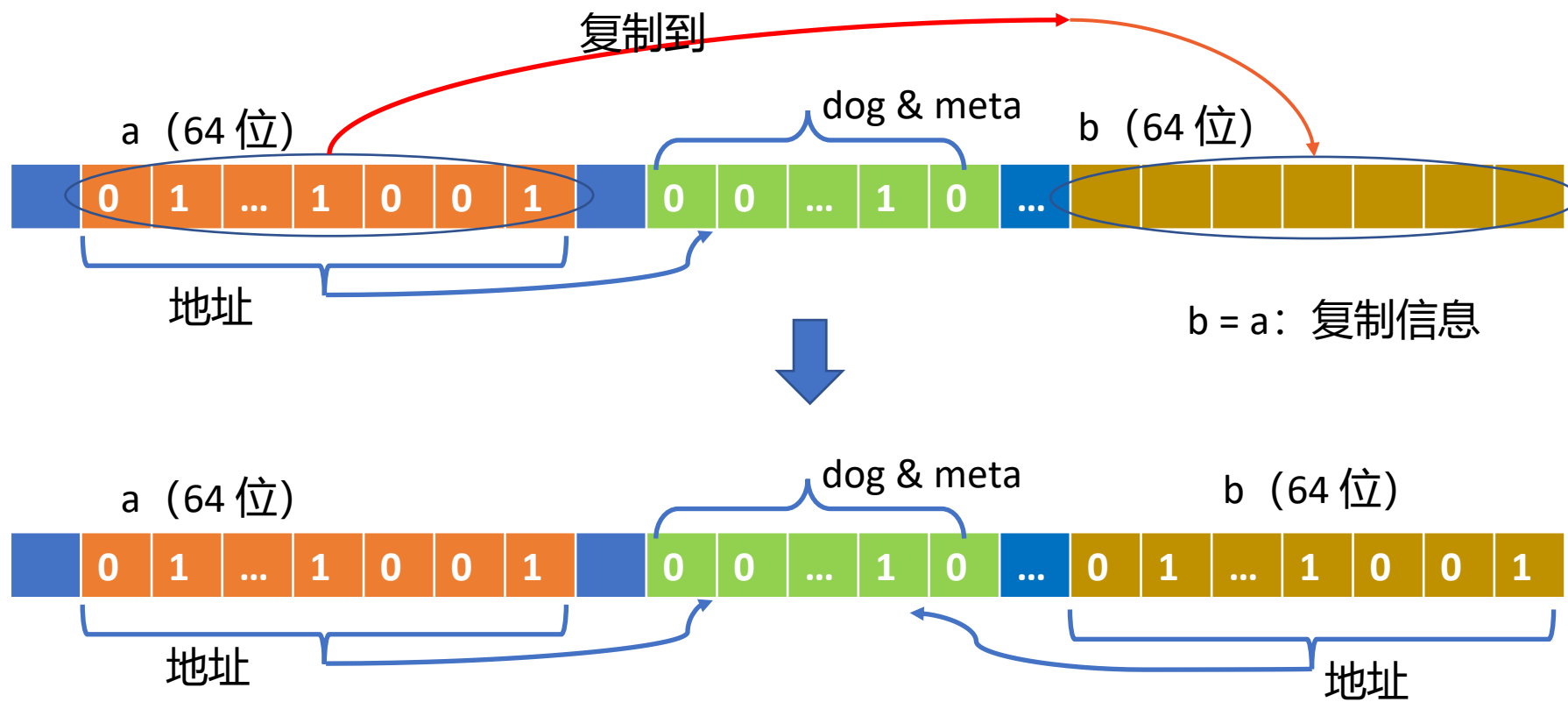
会创建新对象吗? 不会。

```
>>> a = ['a', 'b', 'c']  
>>> b = a  
>>> print(b)  
['a', 'b', 'c']
```

# Python 中的动态类型

问题 6: 把一个变量赋给另一个变量会发生什么?

- 两个变量引用同一内存地址

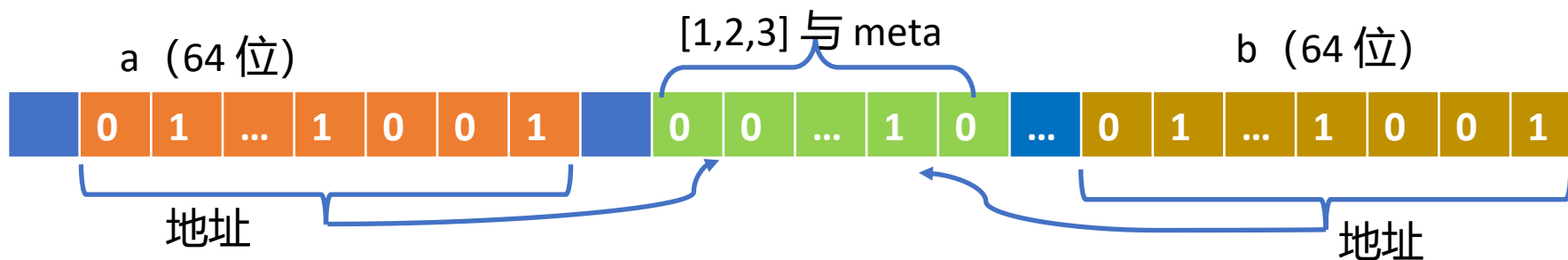


## 可变对象的赋值

将已有对象赋给新变量并不会创建副本  
它只是创建了一个别名

```
01. a = [1, 2, 3]
02. b = a # aliasing
03. b[0] = 0 # update through b
04. print(a) # [0, 2, 3]
05. print(id(a), id(b)) # 4412362176 4412362176
```

id 相同 —— 对 b 的修改会反映到 a 上





## 克隆：创建一份与原对象相互独立的精确副本

- `copy()` 函数

```
01. a = [1, 2, 3]
02. b = a.copy()
03. print(a, b) # [1, 2, 3] [1, 2, 3]
04. b[0] = 10
05. print(a, b) # [1, 2, 3] [10, 2, 3]
```

- 使用切片也会创建副本

```
01. a = [1, 2, 3]
02. b = a[:] # cloning
03. print(a, b) # [1, 2, 3] [1, 2, 3]
04. b[0] = 10
05. print(a, b) # [1, 2, 3] [10, 2, 3]
```

## 身份运算符

检查两个值（或变量）是否引用内存中的同一对象

| 运算符                     | 说明                                                           |
|-------------------------|--------------------------------------------------------------|
| <code>a is b</code>     | 若 <code>a</code> 和 <code>b</code> 是同一对象则为 <code>True</code>  |
| <code>a is not b</code> | 若 <code>a</code> 和 <code>b</code> 不是同一对象则为 <code>True</code> |

```
01. a = [1, 2, 3]
02. b = [1, 2, 3]
03. a == b # True
04. a is b # False
```

```
01. a = [1, 2, 3]
02. b = a
03. a == b # True
04. a is b # True
```

相等 (==) 并不意味着同一对象 (is)

输出是什么？为什么？

```
1  lst = [[]] * 5
2  lst[0].append(1)
3  print(lst)
```

## ■ 练习：比较数据表示

---

你需要存储一条包含三个字段的学生记录：

```
student_id = "2026001", name = "Alice", score = 95
```

请考虑以下三种表示：

```
record_list = ["2026001", "Alice", 95]
```

```
record_tuple = ("2026001", "Alice", 95)
```

```
record_dict = {"student_id": "2026001", "name": "Alice", "score": 95}
```

你的任务：

1. 编写函数 `total_size(obj)` 以估计每条记录的大小，包括容器及其直接元素。（提示：可使用 `sys.getsizeof()`）
2. 从内存占用、可读性、可变性三个方面比较 3 种表示方式
3. 问题：假设需要存储 10,000 条学生记录。你会选择哪种表示，为什么？
4. 你能否提出一种更省内存、同时保持良好可读性与可变性的表示？（提示：不可使用 NumPy 或 Pandas。考虑 Python 内置或标准库功能。）

## ■ 回顾今天的内容

---

- 基本概念
- 基本数据类型：最基础的构建块（整数、浮点数、布尔值）。
- 运算符：用于操作和比较数据的“动词”。
- 集合：用于组织和存储成组数据（列表、元组、字符串、集合、字典）。
- 类型转换：对按需适配数据、利用不同数据类型特性至关重要。
- 动态类型：在运行时确定变量类型，编程更灵活，但会带来额外开销，并需注意类型相关错误。

为什么需要它们？

- 这些概念是 Python 编程的基石，使你能构建日益复杂而强大的应用

# Python 编程与人工智能

**感谢聆听!**